LAB - DAY 3

Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

Fingerprints and safe password storage



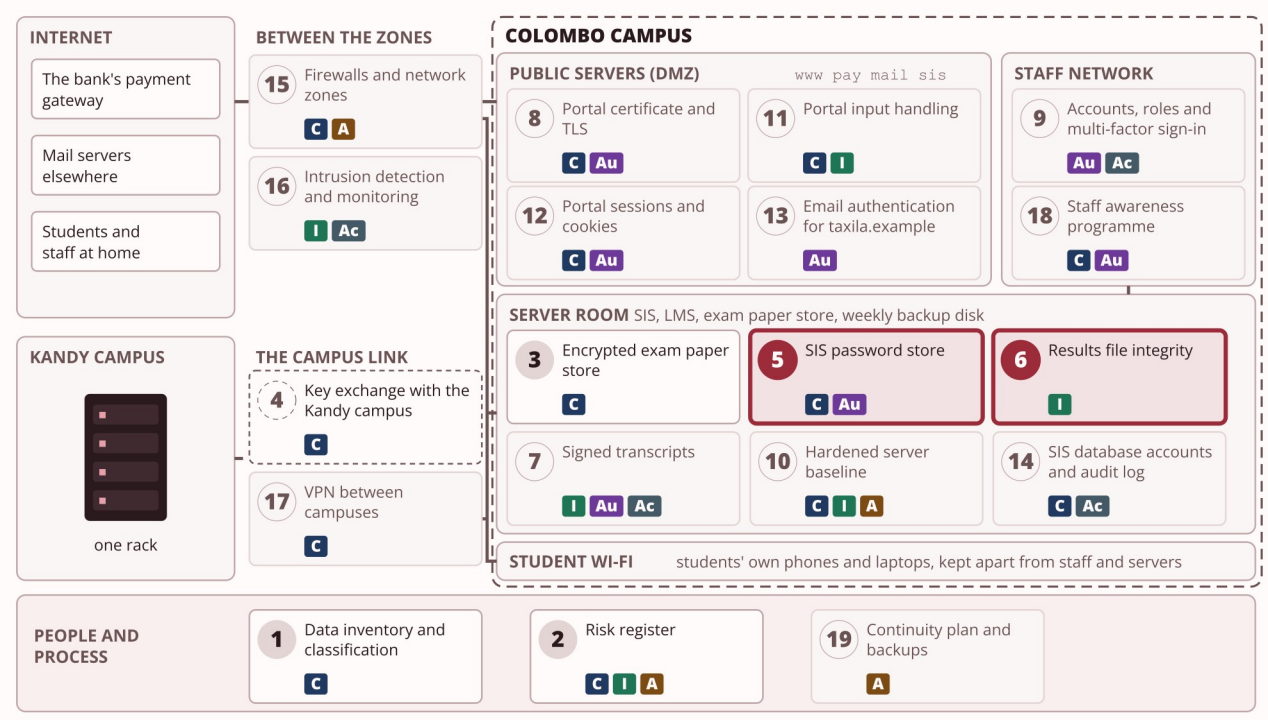
Adds to the register: **5** **6**

Floor tier: every step

Full tier: recommended

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

The security register



TODAY

5 SIS password store

6 Results file integrity

KEY

Added today

Added earlier

Concept only

Not yet added

R

REGISTRAR

If one mark changes in a published results file, I want anyone to be able to see it at once.

The Registrar, at the start of the lab.

Your personal values

```
# results line: your index, a comma, mark = 50 + last digit
COHNDNE999X000,50   (the example belongs to nobody)
# password: the same for everyone in the room
password123
# salts: first and last 8 characters of SHA-256 of your index
5b02a8f8   and   962bb99b
```

No space and no new line after the results line: one extra byte changes the digest.

SEGMENT 1 - 45 MIN

Fingerprint your personal results file

Both tiers

Fingerprint your results file

2 Write your results line: index, comma, mark.

Expected: A two-digit mark from 50 to 59.

3 Take its SHA-256 digest.

Expected: 64 hex characters; the example gives `84a521d5...4b4e3f`.

Floor path: CyberChef `SHA2` with `Size` set to `256`, not the default 512.

4 Add one to the mark and take the digest again.

Expected: Different in almost every character: `b2d1fd0d...e0ef8a`.

Floor path: Type both lines into `avalanche` to count the differences.

The same on the full tier

```
$ printf '%s' COHNDNE999X000,50 > results.csv
$ sha256sum results.csv > results.csv.sha256
$ sha256sum -c results.csv.sha256
results.csv: OK
$ printf '%s' COHNDNE999X000,51 > results.csv
$ sha256sum -c results.csv.sha256
results.csv: FAILED
```

Use `printf`, not `echo`: `echo` adds a line ending and changes the digest.

SEGMENT 2 - 35 MIN

Salt the password store

Both tiers

Salt the password store

5 Hash `password123` with no salt.

Expected: `ef92b778...4473e94f`, the same for everyone.

6 Find your two salts from the SHA-256 of your index.

Expected: First 8 and last 8 characters.

Floor path: `Take bytes`: start 0, length 8; then start 56, length 8.

7 Hash the password with each salt written after it.

Expected: Two digests, unlike each other and your neighbour's.

```
$ printf '%s' password1235b02a8f8 | sha256sum
a0b12530aefa1e72505b5ef0afb82d54e8fc4324...  -
```

SEGMENT 3 - 45 MIN

See why salt and slow hashing protect passwords

Both tiers

Look it up in the table

8 Search the table for the unsalted digest.

Expected: Found at once: `password123`.

Floor path: `salt-shaker`, panel Check a digest.

9 Search the table for both salted digests.

Expected: Not found: nobody computed them in advance.

```
# the first 16 characters are enough to find a line
$ grep ef92b778bafe771e lookup-table.txt
ef92b778...4473e94f  password123
$ grep a0b12530aefa1e72 lookup-table.txt
# nothing printed: not in the table
```

Time a fast hash, then a slow one

10 Time SHA-256 of your salted password.

Expected: A few milliseconds or less.

Floor path: Read the clock and milliseconds under CyberChef's output.

11 Time PBKDF2: 600,000 rounds, your first salt.

Expected: Visibly longer. Example output `e6f049ab...25f9b416`.

Floor path: `Derive PBKDF2 key`: set every box; salt as `UTF8`.

`Derive PBKDF2 key` ignores the input box. Left empty, its salt is random and your output will not match.

The same on the full tier

```
$ printf '%s' password1235b02a8f8 > salted.txt
$ time openssl dgst -sha256 salted.txt
real    0m0.005s
$ time openssl kdf -keylen 32 -kdfopt digest:SHA256 \
    -kdfopt pass:password123 -kdfopt salt:5b02a8f8 \
    -kdfopt iter:600000 PBKDF2
real    0m0.158s
```

Many times slower for one guess. A user waits once; a guesser waits on every guess.

SEGMENT 4 - 25 MIN

Prove it, break it, reset and write up

Both tiers

Prove it, then break it

PROVE IT

- Your two file digests differ in **almost every** character
- Salted and unsalted values differ; all match the recompute tool
- The table finds the unsalted digest, not yours

BREAK IT

- Add one trailing space to the results line and hash it
- A new digest: a space is a byte like any other
- Remove the space; the first digest returns

Reset, and the rules

RESET

- Copy your evidence first, then delete the lab files
- Full tier: restore the `day-3-start` snapshot

RULES OF ENGAGEMENT

- No tool in this lab can affect anyone else's system
- You hash only your own files and passwords and search only the provided table

Record your evidence

EVIDENCE

- Each value goes in the template's table, with the step it comes from
- The sketch's labels and the 120-word question are in the lab sheet
- The two digests of your personal results file, before and after one change
- Your salted and unsalted values for one password
- The table lookup result for the unsalted digest and for your salted one
- The hand-drawn salted-storage sketch
- Your 120-word explanation

Questions & Discussion

Next: Day 4, signatures, certificates and TLS.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision 254abb5